

Frontend Engineer Assignment - Dodo Payments

Submission Deadline - 72 Hours

Build a Tiny Embeddable Checkout

Checkout is where a business actually gets paid. It lives on someone else's website, it has to work first time, and it has to make something complicated feel simple.

We want to see how you build one.

Here's the task:

Build a small checkout that any website can embed. The site adds one script, calls a function, and a checkout opens. You build the script, the checkout itself, and a page that shows it working.

That's it.

We care less about how much you build and more about the choices you make.

What you'll end up with

Three pieces:

1. **The SDK script.** Plain TypeScript, one file a developer can drop into their site. Roughly this shape:

```
DodoCheckout.open({
  productId: "prod_123",
  onSuccess: ({ sessionId }) => {},
  onClose: ({ reason }) => {},
  onError: ({ code, message }) => {},
});
```

2. **The checkout app.** Product, email, card, pay. Fake the payment inside the app, no server needed. This is a small web app of its own, hosted by you, not by the site using it. The customer never leaves the page they were on, yet the card details never touch that page either. The script opens the checkout and stays in touch with it while the customer pays. How that conversation works, and what crosses the line between the two, is yours to figure out.
3. **A demo site.** Pretend it's a real store using your script. A Buy button that opens the checkout right there on the page, and a visible log of the callbacks firing. This is where we'll go to try it.

For the fake payment, use these cards:

- 4242 4242 4242 4242 succeeds
- 4000 0000 0000 0002 declines
- 4000 0000 0000 0341 fails once, then succeeds on retry

Use TypeScript. Beyond that, any framework, library, or tooling you like.

You decide

We've left a lot of this open on purpose.

How much a site can change about the checkout. What the customer sees when a payment fails halfway. What happens if someone hits Buy twice. What the host page should and shouldn't be able to know.

These are your calls. Make them, and tell us why.

A few constraints

1. Have a point of view

Don't build a generic checkout. Look at the flow, question it, and ship the version you think is better. If most checkouts get something wrong, fix it here.

2. Handle the weird states

Payments fail. Networks drop. People click twice. Things don't load. Loading, error, and empty states are part of the product, not an afterthought. Decide what happens in each case, and make sure the host page always finds out the truth.

3. Make it feel finished

One really solid flow is better than ten half-working ones.

The last 10% matters. Focus, keyboard, scroll, motion, copy. The checkout should look like something you'd trust with a card.

Time

Try to keep this to roughly 6 to 9 hours.

We are not expecting production code.

Part of the challenge is deciding what's worth building, and when to build a system versus just shipping the thing. If you ran out of time, tell us what you'd do next instead of doing it.

What to submit

Send us:

- a. A live link to the demo page
- b. Source code
- c. A short README with how to run it and how the pieces talk to each other
- d. Two decisions you went back and forth on
- e. What you'd explore next

Host it wherever you like. A short screen recording of you walking through it would be nice too.

No deck. No case study. Just show us the thing.

What we're looking for

- a. **Taste** - Did you make choices, or defaults?
- b. **UI** - Does the checkout look and feel like something you'd trust with a card? Hierarchy, spacing, typography, motion, and the states in between.
- c. **Judgment** - Where the brief was open, did you make sensible calls and explain them?
- d. **Security** - Did you think about what the host page should and shouldn't be able to do?
- e. **API design** - Is it small, predictable, and hard to misuse?
- f. **Robustness** - Did you handle the weird states, or only the happy path?
- g. **Craft** - Do the small details hold up?
- h. **Ownership** - Does it feel like you shipped a product, or finished an exercise?

What we're not looking for

- a. We are not testing how many browser APIs you know.
- b. We are not looking for the most complicated implementation.
- c. We are not looking for a huge product.
- d. And we are definitely not looking for a giant take-home project.
- e. We want to see what happens when someone who cares about craft gets a rough idea and no spec.

The brief, in one sentence

Build an embeddable checkout you'd be comfortable putting on a stranger's website.